独立审查包 V5

使用说明与注意事项 · 2026-10-31

包名	`Portable-Review-Hooks-V5-20261031.zip`
宿主	Claude Code 2.1.284 宿主（`review_host.py`）；Codex CLI 0.159.2 宿主（`codex_review_host.py`，候选）
依赖	Python 3.10+；你自己的 CLI 登录；一个 OpenAI 兼容的审查模型；一个按接口实现的召回适配器
不依赖	任何特定的记忆系统、信箱或聊天前端

一句话：助手写完一条回复、还没发出去时，宿主先把它拦下。审查模型挑问题，宿主自己去查证据，助手最多改一次，然后才把终稿交给你。

这份 PDF 把随包文档（README、CODEX-README、INTERFACE、PREFETCH、VALIDATION）和完整版审查的接线检查表、守则里适用于本包的部分合在一起。数值以包内代码为准；有出入时以包里同名文档为准。

V5 改了什么

- **模型 ID 同步：**`lib/claude_chat_adapter/models.py` 把 Opus 5.5 的 1M 上下文名字 `claude-opus-5-5[1m]` 认作 `claude-opus-5-5`，和完整版保持一致。**本包宿主不传模型参数、不加载这个文件，所以运行行为没有变化**；同步它只是为了以后有人复用这份代码时不踩坑。
- **新增本 PDF。**另外三处文字改动：README 加了这份 PDF 的链接，VALIDATION 加了 V5 记录，`install.py` 安装时把这份 PDF 一起拷过去。其余代码与 V4 逐字节相同。
- V4 之后完整版的其他审查修正（审查尾巴整理、带图审查证明、信箱交付、Codex Chat 尾巴）都落在本包没有的部分：本包每轮新开会话、没有换窗、没有信箱，所以不需要移植。

一轮是怎么走的

1 · 回复	2 · 拦下	3 · 挑问题	4 · 封工具、查证据	5 · 改一次
助手照常回复，审查开始前可以用只读工具	完整草稿一交出，Stop hook 拦住，先不发	审查模型只列问题，不动手改稿	宿主封住工具，自己去召回接口查历史，把证据递回	助手改一次或原样保留（KEEP），宿主核对后输出

stdout 只在终稿通过绑定校验后输出一条 JSON：`content` 是完整终稿，`review` 是审查结果。草稿、KEEP 控制记录、证据包都不会当正文输出。

接线检查表

装好后逐项打勾。标 必接 的漏一项，审查就等于没开。

用宿主启动，不是只拷 Stop 脚本 必接 装机时

两版都是独立命令行宿主。宿主每轮自己生成 `PreToolUse` / `PostToolUse` / `Stop` 设置，把子进程的全部公开输出留在宿主内，绑定本轮输入、会话、草稿和终稿事件。

> **常见漏法：**把 Stop 脚本写进自己的 `~/.claude/settings.json` 或项目 hook，以为就有审查了。没有宿主的事件绑定和封条，草稿照样会直接发出去。Codex 的原生 JSON 流本身会先吐出原稿，去掉宿主就不能再说「发送前拦截」。

锁定 CLI 版本 必接 装机时

宿主启动时核对 `--version`：Claude Code 认 `2.1.284`（写在 `config.json` 的 `cli_version`），Codex 认 `0.159.2`。版本不符直接不启动。

> **常见漏法：**CLI 自动更新。新版本的公开事件、Stop 或 PreToolUse 格式一变，草稿绑定就对不上，这一轮失败。升级前先在隔离环境把审查跑一遍，再改版本号。

审查模型 必接 装机时

用宿主进程的环境变量配置，**不要把密钥写进共享文件**；密钥会从 CLI 子进程里移除。

```
export PORTABLE_REVIEW_MODEL_BASE_URL='https://你的模型服务地址'
export PORTABLE_REVIEW_MODEL_NAME='你的审查模型名'
export PORTABLE_REVIEW_MODEL_API_KEY='你的审查模型密钥'
# 可选：模型支持关闭思考时
export PORTABLE_REVIEW_THINKING=disabled
```

要求：公网 HTTPS、443 端口，支持 `chat/completions` 的 JSON 模式。审查模型独立于 Claude 订阅，按它自己的服务商计费。每轮最多分类 1 次。

> **建议：**选响应快的模型，能关思考就关。测试里有一次默认开思考的调用撞到 20 秒上限，被记成 unavailable。

召回适配器 要查历史就必接 装机时

```
export PORTABLE_RECALL_URL='http://127.0.0.1:18889/recall'
# 需要时：export PORTABLE_RECALL_TOKEN='...'
```

本机可以用 loopback HTTP，公网地址必须 HTTPS。协议见 `INTERFACE.md`，要点：

- 宿主每轮最多 POST 一次，1–4 条查询；等 8 秒，不重试。
- 返回 `ok`（1–12 条证据）/ `no_match` / `unavailable`。每条证据的正文最多 4096 字节，整个响应最多 64 KiB。
- 必须原样回显 `session_id` / `turn_id` / `draft_id` 和版本，并给出原始请求体字节的 SHA-256。对不上就算不可用，用来防止串轮。
- **适配器必须只读。**断开连接不代表远端停止了，所以一次检索不能触发任何写入。
- 身份、权限、禁查范围由适配器自己判断。请求里的 `recall_scope` 和各种 ID 都不是授权凭据，拿不准就返回 `unavailable`。
- 保留来源原来的日期，不知道就填 null。不要截断以后冒充完整原文，也不要把助手以前的计划当成已经发生的事。

附带的 `example_recall.py` 是单用户、只监听本机的 JSONL 字面查找示例，用来说明接口怎么实现。它不是记忆库，也不是语义召回。

本轮预取（可选） 装机时

已经有可信的召回服务的话，配置 `PORTABLE_PREFETCH_URL`（以及可选的 `PORTABLE_PREFETCH_TOKEN`）。宿主在主回复前取一份本轮材料，同时交给主模型和审查分类器。之后审查对同一 scope、规范化后完全相同的查询直接复用，不再查第二次。最多 4 项，材料包最多 6000 字节，等待 8 秒。出任何问题都整份不交付、不复用，原来的召回路径照常可用。详见 `PREFETCH.md`。

冰箱贴（可选） 用户维护

冰箱贴是用户写给助手的固定提醒。新安装一张都没有，全部由用户自己写，助手这边没有写冰箱贴的工具。

```
python3 review_host.py --state ./state notes
python3 review_host.py --state ./state add-note --trigger '触发词' --note-file ./note.txt
python3 review_host.py --state ./state edit-note --id RULE_ID --version 1 --note-file ./not
e.txt
```

- 正文最多 180 字，原样保存、原样交给助手，不改写也不摘要（只按规则裁掉首尾空白）。每个触发词最多 80 字，最多 128 张。
- 修改时要提交你看到的那个版本号，版本冲突会被拒绝。
- 草稿命中并违反的冰箱贴，每轮最多 2 张，跟唯一一次修订材料一起交给助手，不会多起一次模型调用。

Codex 的只读目录（可选） Codex 宿主

`config.codex.json` 里的 `read_root` 给助手开放 `read` / `list` / `search`：只能用相对路径，逐段拒绝符号链接和 `..`。单个文件和搜索结果最多 4096 字节，每个目录最多列 100 个名字，每轮最多调用 24 次。只选你愿意让 Codex 读、也愿意发给审查模型的目录；不需要就删掉这一行。需要 Linux pidfd。状态目录必须属于当前用户、没有组和其他人权限，并且放在 Git 工作树之外。

写死的规矩，以及为什么

下面这些是故意写死的。装上以后觉得哪里「太死了」，先看这里。

01 审查一开始，本轮不能再调用任何工具

现在 草稿一进审查，宿主就在 SQLite 账本里给这一轮记上 `tools_sealed`。之后每次 PreToolUse 都查这道封条，**审查前已经获准的工具调用 ID 也不能重放**。本包没有信箱，所以没有任何例外。

为什么

- 审查审的是这一份草稿。改稿时要是还能调工具，助手就会拿新查到的东西写出新断言，可这些断言没人审过。
- 只改一次才有尽头，不会「改稿 → 调工具 → 再审 → 再改」转好几圈，让用户一直看着空白。
- 查证据是宿主的活。需要翻历史的地方，宿主在审查阶段已经查好，证据直接递给助手。
- 封条记在宿主的数据库里，不是写在提示词里求模型自觉。

> **会踩坑：**草稿里写「我这就去查一下」，审查一过助手真去调工具，就会撞上封条，这一轮失败（下一轮照常）。**建议在人格或系统提示里写清楚：要用的工具先用完，再写草稿；草稿里不要承诺「待会去查」。**

Codex 官方说明有些工具路径不经过 hook，hook 超时或出错也可能放行。所以 Codex 版不要把 hook 说成绝对封条。

02 只改一次，改完不再审

现在 有具体问题才发一次修订材料，说明里写明「这是本轮唯一一次核对或改稿」。下一条公开消息必须和这次 Stop 绑定在同一轮、换一个消息 ID，最多一次。选 KEEP 时，宿主自己把原稿还原回来，不让助手重写一遍。

为什么 保证延迟有上限、不打转。

03 三道不靠审查模型的硬规则

现在 审查模型也会漏。下面三类句子长相固定，宿主用规则直接抓，一命中就强制查一次历史：

- 说「不记得」「没印象」「找不到」：拿说忘了的那个名字去查。
- 承认「是猜的」「没去翻」「还没查」：拿猜的那件事去查（每轮最多 2 条）。
- 反问用户早就说过的稳定事实，比如宠物的品种、名字、生日、毛色、长相、年龄：拿名字去查，查到答案就别再问（每轮最多 2 条）。

> **会踩坑：**自己改规则时，查询只用名字，不要加「品种」这类属性词。加了以后，检索就不把它当名字查了，答案可能挤不进返回的材料里。

不会误伤 规则只认人名和代词。「你今天想吃什么」这种普通问句，以及用户这一轮才第一次提到的新对象，都不会命中。用户和助手的称呼、别名、宠物名写在安装时生成的 `persona.local.json` 里，默认是「用户 / 助手 / TA」和空别名，按需要自己改。

04 每一步都有时限

环节	上限	超时怎么算
Stop hook（Claude 宿主）	38 秒	超时只算「没核到」，**不算「查过、没有」**。草稿不会因为超时被改成不确定的说法，也不能借超时编出相反的事实。
审查分类，单次请求	20 秒；同时最多 2 个；回包最多 128 KiB；不重试	
召回 / 预取	各 8 秒，不重试	
整轮（含 CLI 运行）	默认 180 秒，`turn_timeout_seconds` 可设 5–900	

为什么 审查卡在用户和回复之间，多等一秒用户就多看一秒空白。整轮超时会杀掉本轮整个进程组、撤销回调，把这一轮记为失败。

05 不可用 ≠ 通过；没查到 ≠ 没发生

分类或召回不可用时，宿主保留绑定的原稿，并标记 `unavailable`。这不等于审查通过。`no_match` 和 `unavailable` 分开记：没查到不代表那件事从来没有发生过。

06 失败只影响这一轮

CLI 超时、撞封条、没有 Stop、草稿绑定不符，都只把这一轮记为失败，不输出草稿，也不自动重放。下一条消息照常发。同一个 `state` 目录有单 owner 文件锁，不能同时跑两个宿主。

07 只带最近两轮，不是完整聊天记录

每次运行都新建一个原生会话，带上最近两轮**已完成**的原话。单条超过 8192 字节，或合计超过 16384 字节时，整条省略并记下省略了几条，不会截断以后冒充原话。它不会读 CLI 历史、隐藏思考或全量聊天记录，也没有自动换窗（slim）。

08 审查材料不留在会话里，但会发给审查模型

Claude 子进程用不持久化的原生会话。Codex 每轮用独立的临时 home 和空工作目录，本轮结束、进程组被证明全部退出以后，整轮目录（包括 SQLite/WAL）一起删掉。宿主和看门进程同时异常退出时，资料可能留到下一次启动才清。下一轮只带已完成的用户原话和终稿，不带审查包。

这不是安全擦除：系统缓存、交换空间、备份和模型服务商那边的副本，本包管不到。

隐私与数据

- `state/` 里有原始对话和回执，是私人数据。保护好它，别打包分享，也别用旧的状态库覆盖新对话。
- 审查会把当前用户输入、完整草稿和预算内的最近上下文发给你选的审查模型；需要历史时，查询会发给召回适配器。只用你信任的服务。
- 召回来源的授权、准确性和作用范围，由你自己的适配器负责。回显和哈希只能防串轮，不能证明第三方材料是真的。
- 本包不携带任何人的聊天、记忆、账号、密钥或已填写的画像；冰箱贴和称呼从空模板开始。

本包没有的（完整版才有）

完整版的功能 / 规矩	本包的情况
接 Brain 的审查中转、审查提示词哈希三处一致	不接任何记忆系统。审查模型和召回接口由你自己配置，没有三处哈希要对。
官端信箱（pending）与审查期间暂存信箱意图	没有信箱，封条也没有任何例外。
自动换窗（slim）：原话逐字节核对、触发线与预算、审查材料剥离、换窗时排除失败片段	没有换窗。每轮新开会话，只带最近两轮已完成原话。
网页 / 手机界面、冰箱贴点赞点踩和静音、每周命中导出	只有命令行。冰箱贴通过 `notes` / `add-note` / `edit-note` 管理。
Bash、写文件、任意 MCP、附件上传、记忆写入	Claude 宿主只开放 Read / Glob / Grep；Codex 宿主只开放可选的只读目录。

它不是丢进任何现有聊天前端就能用的插件。新应用要内嵌的话，得实现和本包宿主等价的事件绑定、封条和状态机，不能只拷 Stop 脚本。

安装与第一次跑

```
python3 -B verify_manifest.py           # 先校验，应显示 Verified 60 files
python3 install.py --prefix "$HOME/portable-review"
cd "$HOME/portable-review"
cp config.example.json config.json      # 把 claude_command 指向自己的 Claude
# 设好上面的审查模型 / 召回环境变量后：
printf '%s' '本轮消息' > message.txt
python3 review_host.py --state ./state run --config ./config.json \
  --session my-conversation --prompt-file ./message.txt --cwd ./readable-project
```

安装器不会下载程序、用 sudo、写系统目录、碰已有数据库或启动服务；目标目录已经存在就拒绝安装。Codex 版的步骤见 `CODEX-README.md`。

已验证 / 未验证

已验证（隔离、合成数据）	未验证
• 真实 Claude Code 2.1.284 跑五轮：直接通过、唯一一次续答、KEEP 还原、历史接口收到查询 • 真实 Codex 0.159.2：通过、修订、KEEP、审查 503 保原稿、无匹配、冰箱贴、超时恢复、只读工具越界拒绝、SIGKILL 后清理 • 真实 OpenAI 兼容审查模型用虚构内容单独测过三类路线 • 两端各两轮预取和同题复用，没有二次召回 • V5：包内三组测试、宿主状态机 18 项、模型 ID 6 项检查通过	• 接收者自己的账号、审查服务商、真实记忆库的整条链路 • 其他 CLI 版本、操作系统、认证方式、原生 TUI • 多用户或公网开放场景 • 审查不保证事实正确，效果取决于模型和记忆系统

运维小规矩

☐ 先在自己的隔离环境接线验证，再用于真实聊天。
☐ 升级 CLI 前，先隔离把审查跑一遍，再改 `cli_version`。
☐ 保留旧的安装目录，回退时切回旧程序入口；不要回滚或覆盖 `state` 数据库。
☐ 测试不要整库拷贝聊天数据：用只读方式打开，或者只拷需要的表，用完就删。
☐ 出了失败，看一下状态，再新发一轮；不要自动重放之前的请求。

文件清单和校验值见包内 `MANIFEST.sha256.json`，解压后运行 `python3 -B verify_manifest.py`。